

SwiftSign: An AI-Powered Classroom Communication Assistant for Hearing-Impaired Students

Raneen Yusoh (2311160018 / SEC 2)

Tunchanoke Kongpon (2311130009 / SEC 3)

Chonlada Saetang (2311010004 / SEC 3)

Information Technology, Stamford International University

ITE233 - Introduction to Internet of Things

Lecturers: Aj. Zakariyya Bature & Aj. Atikom Srivallop

June 23, 2026

Contents

Abstract.....	3
Feasibility Study	4
Technical Feasibility	4
Economic Feasibility.....	4
Operational Feasibility	5
Schedule Feasibility	5
Analysis and Design	6
Hardware Components.....	6
Software Tools.....	6
Logical Design of the System	6
System Components and Logical Roles	7
Physical Design of the System (Connections).....	8
Structured Chart.....	10
Implementation	11
Hardware Implementation.....	11
Software Implementation.....	13
Outcome.....	20
Summary.....	22
References.....	24

Abstract

Communication plays a crucial role in the learning process; however, hearing-impaired students often face challenges in classroom environments where communication primarily depends on spoken language (Ismail et al., 2023). These barriers can limit classroom participation, reduce interaction with teachers and peers, and affect the overall learning experience. To address this issue, this project presents SwiftSign: An AI-Powered Classroom Communication Assistant for Hearing-Impaired Students, an Internet of Things (IoT)-based system designed to improve communication accessibility and support inclusive education. The proposed system integrates various IoT devices and sensors to facilitate real-time communication. An ESP32-CAM module is used to capture and recognize predefined sign language gestures, while an ESP32 DevKit serves as the main controller for processing data and managing system operations. Once a gesture is recognized, the corresponding message is displayed on an OLED display and converted into speech through a DFPlayer Mini, MicroSD card, and 3W speaker. This functionality enables hearing-impaired students to communicate more effectively with teachers and classmates (Maashi et al., 2025). To enhance system performance, an HC-SR04 ultrasonic sensor measures the distance between the user and the device, while a PIR motion sensor detects user presence and movement. These sensors allow the system to respond automatically and improve energy efficiency. In addition, the system is connected to the Blynk platform, enabling wireless monitoring and control. This project demonstrates the application of IoT concepts, including sensor integration, embedded systems, wireless communication, and real-time data processing. By combining affordable hardware components with intelligent communication features, SwiftSign aims to reduce communication barriers, promote active classroom participation, and create a more accessible learning environment for hearing-impaired students.

Feasibility Study

The feasibility of SwiftSign: An AI-Powered Classroom Communication Assistant for Hearing-Impaired Students was assessed to determine whether the project can be successfully developed and implemented using the available resources, budget, and timeframe. The evaluation focuses on technical, economic, operational, and schedule feasibility.

Technical Feasibility

SwiftSign is technically achievable because the required hardware and software technologies are accessible and compatible. The system is built using an ESP32 DevKit as the main controller and an ESP32-CAM for capturing sign language gestures. Other components, such as the OLED display, DFPlayer Mini, speaker, HC-SR04 ultrasonic sensor, and PIR motion sensor, support communication and user interaction. The Blynk platform provides wireless connectivity and remote monitoring capabilities. Since these technologies are widely used in IoT applications and supported by extensive documentation, the project can be developed without major technical difficulties.

Economic Feasibility

The proposed system is cost-effective because it relies on affordable and readily available components. Most of the hardware required for development can be purchased at a reasonable price, making the project suitable for an academic environment. Furthermore, the use of open-source software and development tools helps reduce implementation costs. As a result, the project can be completed within a limited budget while still delivering its intended functionality.

Operational Feasibility

SwiftSign is designed to be practical and easy to use in classroom settings. The system allows hearing-impaired students to communicate through recognized sign language gestures, which are translated into text and audio outputs. Features such as automatic motion and distance detection improve convenience and system responsiveness. These functions enable effective communication between students, teachers, and classmates, supporting a more inclusive learning environment.

Schedule Feasibility

The project is expected to be completed within the allocated academic semester. A development plan consisting of system design, hardware integration, programming, testing, and documentation ensures that all tasks can be completed on schedule. The availability of components and development resources further contributes to the project's timely completion.

Analysis and Design

Hardware Components

Components	Quantity	Price (THB)	Purpose
ESP32 DevKit	1	180	Main controller for processing data and controlling outputs
ESP32-CAM	1	220	Captures sign language gestures
PIR Sensor	1	70	Detects user presence
HC-SR04 Ultrasonic Sensor	1	60	Measures distance between user and device
OLDE Display	1	280	Display recognized phrases and system messages
DFPlayer Mini	1	120	Play pre-recorded audio
Speaker	1	50	Outputs speech audio
MicroSD	1	60	Stores audio files
Breadboard	1	80	Circuit prototyping
Jumper Wires	Multiple	80	Connect system components
USB Cable	2	100	Used to upload code and provide power to the ESP32 boards.

Software Tools

Software	Purpose
Arduino IDE	Writing and uploading code to ESP32
Blynk Application	Monitoring and controlling the system remotely
Python	Gesture recognition and classification

Logical Design of the System

Features:

- Sign language gesture recognition
- Real-time phrase display on OLED
- Audio playback of recognized phrases
- User presence detection using PIR sensor
- Distance measurement using ultrasonic sensor

- Wireless communication between ESP32-CAM, Python model, and ESP32 Remote monitoring through Blynk

Objectives:

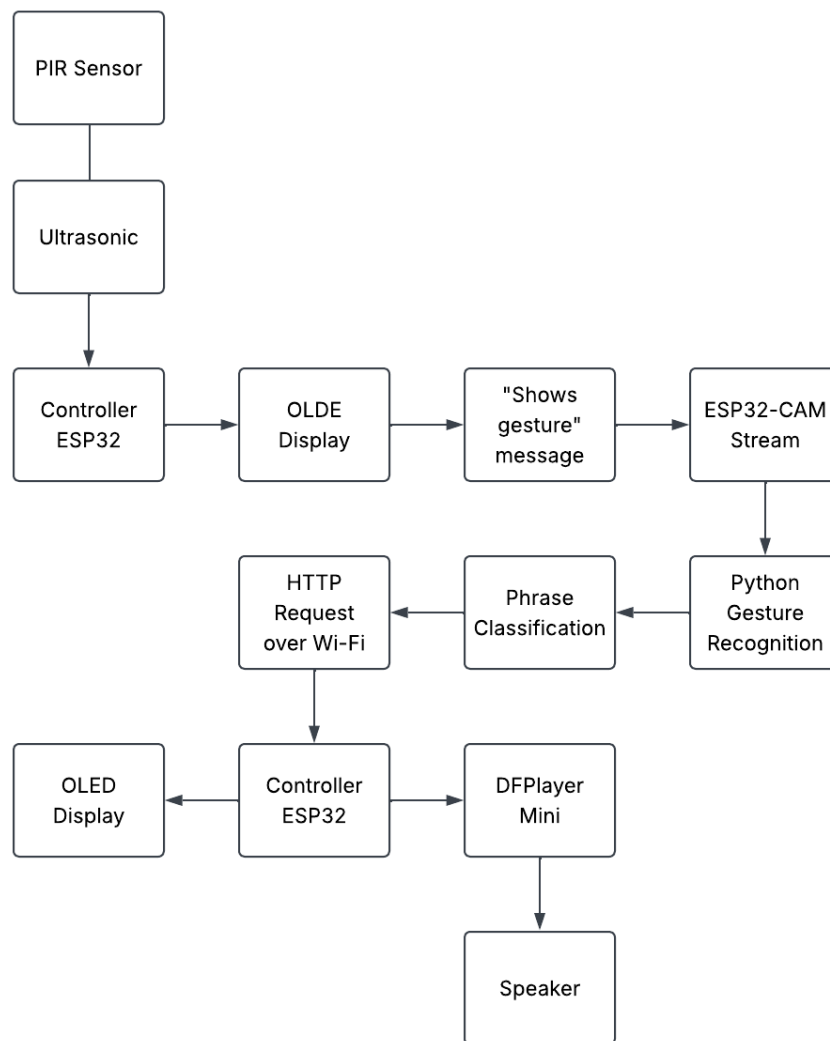
- To convert sign language gestures into text and speech
- To assist communication between hearing impaired and non-sign language users
- To provide real-time recognition and feedback
- To create an affordable and portable assistive device

System Components and Logical Roles

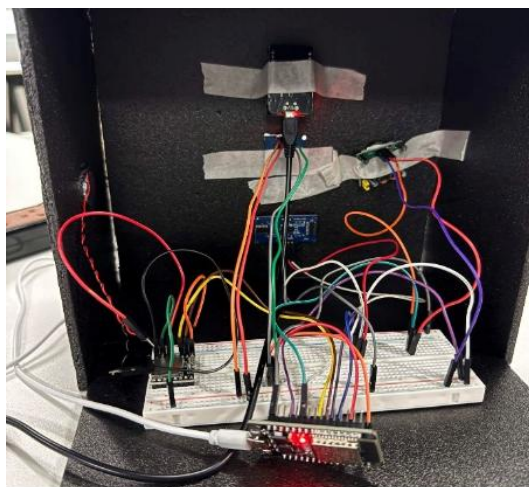
Component	Logical Role	Description
ESP32 DevKit	Central Controller	Controls system operations and outputs
ESP32-CAM	Input Device	Captures hand gestures
PIR Sensor	Input Sensor	Detects user presence
HC-SR04	Input Sensor	Measures distance
Python Recognition Module	Processing Unit	Recognizes and classifies
OLED Display	Output Device	Display phrases
DFPlayer Mini	Audio Module	Plays audio files
Speaker	Output Device	Produces speech

Logical Workflow of the SwiftSign System

The logical workflow diagram shows how the different components of the SwiftSign system work together, from user detection and gesture recognition to the final text and audio outputs.



Physical Design of the System (Connections)



The following table summarizes how the hardware components are connected within the Sign Language Translation System

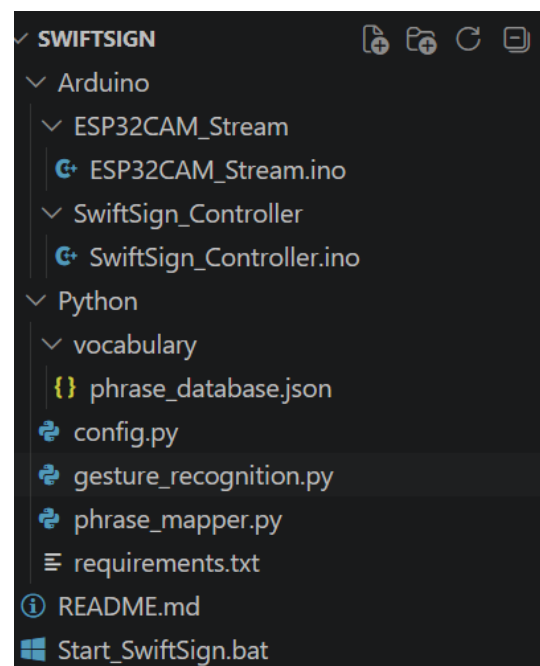
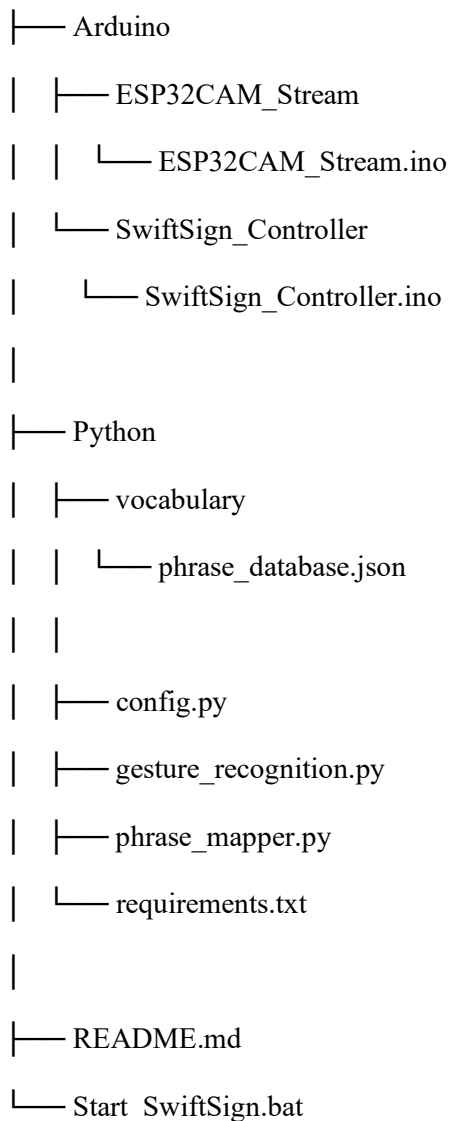
Component	VCC	GND	Signal Pins	Description
PIR Sensor	3.3V	GND	GPIO 27	Detects the presence of a user near the device.
HC-SR04 Ultrasonic Sensor	5V	GND	TRIG → GPIO 5, ECHO → GPIO 18	Measures the distance between the user and the system.
SH1107G OLED Display	3.3V	GND	SDA → GPIO 21, SCL → GPIO 22	Displays system statuses and recognized phrases.
DFPlayer Mini	5V	GND	RX → GPIO 16, TX → GPIO 17	Controls audio playback.
Speaker	—	—	SPK1, SPK2	Plays the recognized phrase as speech.
ESP32-CAM	5V	GND	Wi-Fi Communication	Captures hand gestures and streams video for recognition.
MicroSD Card	—	—	Connected to DFPlayer Mini	Stores prerecorded audio files.
PIR Sensor	3.3V	GND	GPIO 27	Detects the presence of a user near the device.
HC-SR04 Ultrasonic Sensor	5V	GND	TRIG → GPIO 5, ECHO → GPIO 18	Measures the distance between the user and the system.
SH1107G OLED Display	3.3V	GND	SDA → GPIO 21, SCL → GPIO 22	Displays system statuses and recognized phrases.

The ESP32 acts as the central controller of the system. It receives input from the sensor, communication with the gesture recognition module, and controls the display and audio output device

Structured Chart

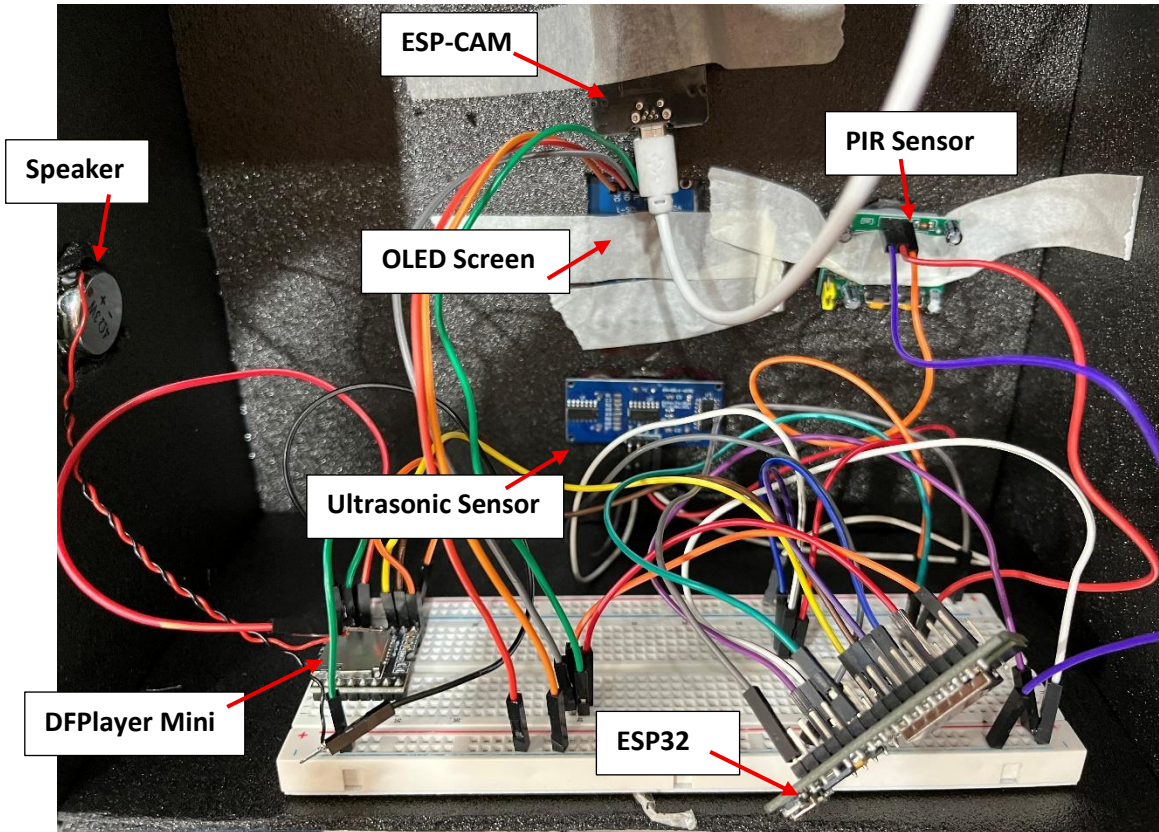
The structured chart shows the hierarchical breakdown of the system into functional modules. Each module is responsible for a specific operation, such as detecting the user, recognizing sign language gestures, processing data, and generating text and audio outputs. This design enhances clarity, modularity, and ease of debugging.

SWIFTSIGN



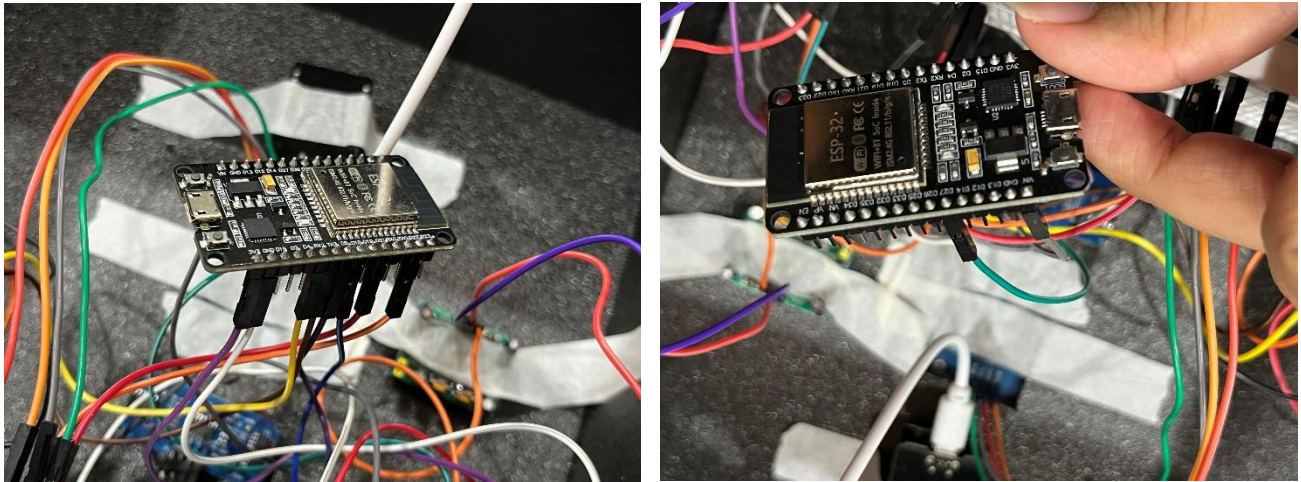
Implementation

Hardware Implementation



The controller ESP32 acts as the central unit of the system. It is responsible for receiving data from the sensors and then deciding what actions should be taken (Espressif Systems, 2025). It also communicates with the Python application, updates the OLED display, and controls audio playback through the DFPlayer Mini.

ESP32 Controller



When the system is powered on, it will first try to determine whether somebody is in front of it. This is done using a PIR sensor connected to GPIO 27. The sensor detects changes in infrared radiation that occur when a person moves within its field of view. Once movement is detected, the sensor outputs a HIGH signal to the controller ESP32.

Next is determining whether the user is close enough to use the system. This is done using the HC-SR04 ultrasonic sensor connected to GPIO 5 and GPIO 18. The ESP32 sends an ultrasonic pulse and waits for the sound wave to return after bouncing off an object. The time taken for the sound wave to return is then converted into a distance value. The formula used is $\text{Distance (cm)} = (\text{Duration} \times 0.034) \div 2$. The value 0.034 represents the approximate speed of sound in centimeters per microsecond. If the user is within 50 cm of the system, the OLED displays "Show gesture" and the system becomes ready for gesture recognition. Otherwise, the OLED displays "Move closer".

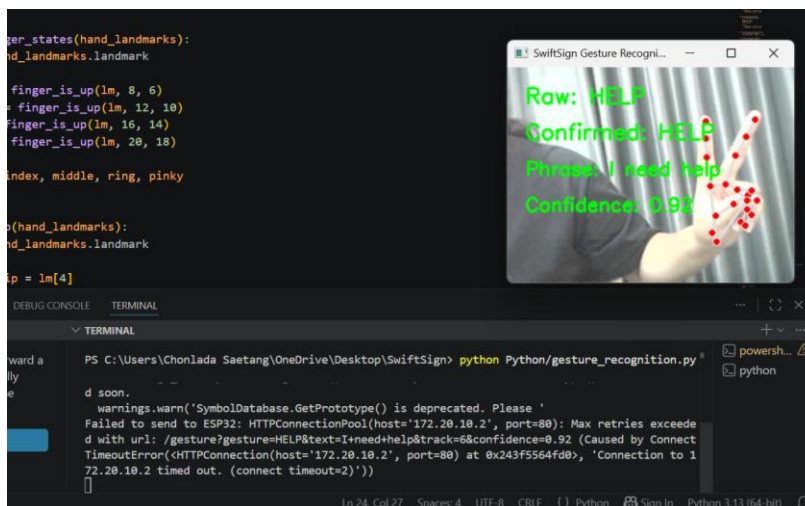
The system also uses a 1.5-inch SH1107G OLED display to provide visual feedback to the user. Messages such as "Waiting", "Move closer", and recognized phrases are displayed on the screen. Audio feedback is provided by a DFPlayer Mini and speaker. Once a phrase has been

recognized, the controller ESP32 displays the phrase on the OLED and plays the corresponding audio file.

Software Implementation

The ESP32-CAM does not perform the gesture recognition itself. Rather, its main purpose is to provide a continuous video feed that can be processed by the Python script. The ESP32-CAM continuously streams video frames over Wi-Fi to Visual Studio, which is running the Python script that facilitates gesture recognition. The application uses Google's MediaPipe Hands framework to detect and track 21 hand landmarks in real time (Lugaresi et al., 2019). These landmarks represent important points on the hand, such as the fingertips and finger joints. The script receives the footage from the ESP-CAM and generates the landmark data used to determine which fingers are extended and classify the corresponding hand sign. The final prototype supports eight classroom communication phrases. These phrases are Hello, Goodbye, Yes, No, I have a question, I need help, May I go to the restroom, and Thank you.

Gesture Recognition Interface in Visual Studio



Python Libraries Used for Gesture Recognition

Library	Purpose
cv2 (OpenCV)	Processes video frames and displays the gesture recognition interface.
mediapipe	Detects and tracks hand landmarks for gesture recognition.
requests	Sends recognized phrases to the controller ESP32 through HTTP requests.
collections	Stores recent predictions and supports gesture confirmation logic.
time	Provides timing and delay functions during program execution.
json	Loads phrase data from the phrase database file.
pathlib	Handles file paths and accesses the phrase database file.

On the hardware level C++ is the coding language, developed in the Arduino IDE. Its usage is split into two main parts. The first part is the ESP32-CAM code, which connects the camera module to Wi-Fi and creates the video stream. The second part is the SwiftSign Controller code, which acts as the main IoT program of the system.

The SwiftSign Controller code is responsible for reading sensor values, updating the OLED display, communicating with Blynk, receiving recognized phrases through HTTP requests, and triggering audio playback. Additionally, several libraries were used during development to simplify integration between the hardware and software components. These libraries provided functionality for various components reducing the amount of code that needed to be developed from scratch.

C++ Libraries Used for ESP32 & ESP-CAM

Library	Component	Purpose
WiFi.h	ESP32-CAM, Controller ESP32	Connects the ESP32 devices to the Wi-Fi network.
esp_camera.h	ESP32-CAM	Controls the camera module and captures video frames.
esp_http_server.h	ESP32-CAM	Creates the web server used to stream video frames.
WebServer.h	Controller ESP32	Creates HTTP endpoints for communication between the Python application and ESP32.
BlynkSimpleEsp32.h	Controller ESP32	Connects the ESP32 to the Blynk dashboard and enables remote control functions.
Wire.h	Controller ESP32	Facilitates I2C communication between the ESP32 and OLED display.
Adafruit_GFX.h	Controller ESP32	Provides graphics and text rendering functions for the OLED display.
Adafruit_SH110X.h	Controller ESP32	Controls the SH1107 OLED display module.
DFRobotDFPlayerMini.h	Controller ESP32	Controls audio playback through the DFPlayer Mini module.

Ultrasonic Distance Calculation in the SwiftSign Controller Code

```

float getDistanceCM() {
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);

    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);

    long duration = pulseIn(ECHO_PIN, HIGH, 30000);

    if (duration == 0) {
        return -1;
    }

    return duration * 0.034 / 2;
}

```

The program above measures how long it took for the sound wave to return after bouncing off an object. In this case, the duration value is converted into centimeters and used to determine whether the user is within the acceptable operating range.

The controller code also contains a web server that receives recognized phrases from the Python application. Once phrase data is received, the controller updates the OLED display and plays the corresponding audio file.

Phrase Handling in the SwiftSign Controller Code

```
void showPhrase(String face, String message) {
  display.clearDisplay();
  display.setTextColor(SH110X_WHITE);

  drawCuteFace();

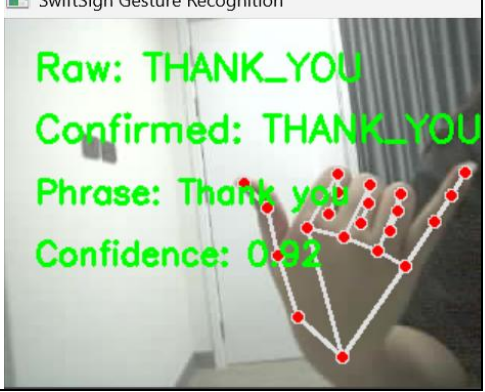
  display.setTextSize(1);
  display.setCursor(8, 92);

  if (message.length() <= 18) {
    display.println(message);
  } else {
    display.println(message.substring(0, 18));
    display.setCursor(8, 106);
    display.println(message.substring(18, 36));
  }

  display.display();
}
```

Hand Gesture Vocabulary and Corresponding Phrases

Hand Sign	Phrase	OLED Screen	Gesture Recognition Interface
Open palm (fingers spread)	Hello		
Open palm (fingers together)	Goodbye		
Thumbs up	Yes		
Thumbs down	No		

Index finger	I have a question		
Peace sign (index and middle fingers)	I need help		
Three fingers (index, middle, and ring fingers)	May I go to the restroom?		
Pinky and thumb extended	Thank you		

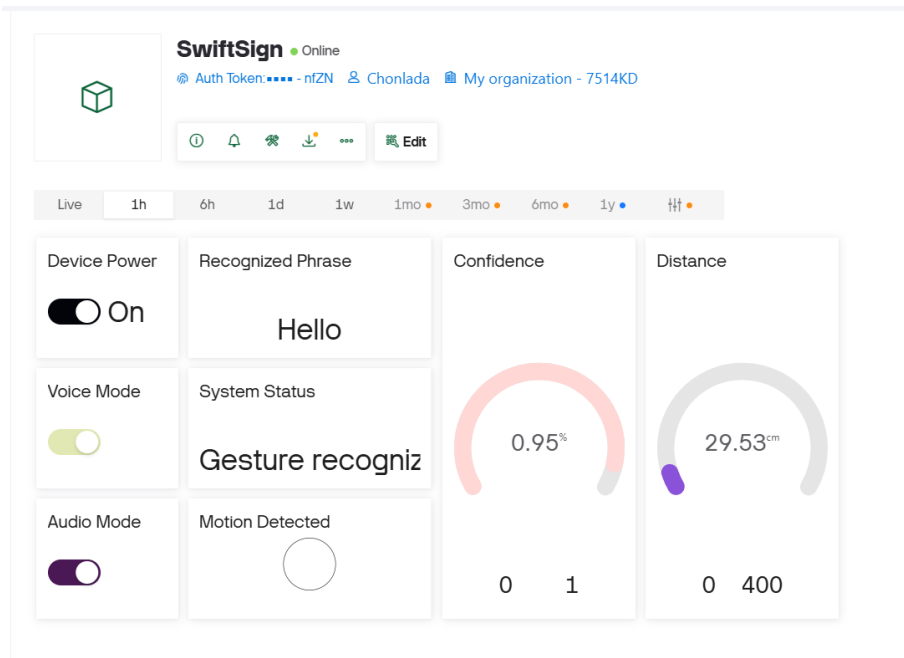
Closed fist	Reset	(No output is produced on the OLED, rather the system unlocks phrase hold)	
-------------	-------	--	--

However, hand gestures are not always detected perfectly. More so, the gesture recognition often recognizes multiple signs despite being consistently shown a single hand sign. To address this, the system stores recent predictions and only confirms a gesture after it has been detected repeatedly. So whichever hand sign was shown the most within 15 frames gets confirmed by the program and triggers the audio and OLED text output. Once a phrase has been successfully recognized, the system enters a locked state. In this case, the phrase is only spoken and displayed once, even if the user continues holding the same hand sign. This was implemented because the system would otherwise continuously repeat the same phrase. The system remains in this state until a closed fist reset gesture is performed.

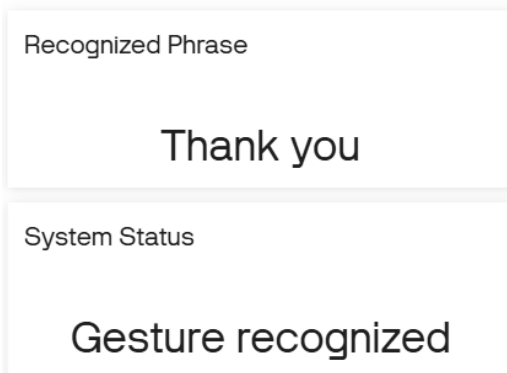
The controller code also includes the Blynk functions used to control the system remotely. Through the dashboard, users can turn the device on or off, switch between male and female voice modes, and enable or disable audio playback (Blynk, 2025).

Code Connecting ESP32 to Blynk

```
#define BLYNK_TEMPLATE_ID "TMPL655Y65KWr"
#define BLYNK_TEMPLATE_NAME "SwiftSign"
#define BLYNK_AUTH_TOKEN "Av04klvBs6NVbUUcMUBG9reMgtP8nfZN"
```

Blynk Dashboard for Remote System Control**Outcome**

During testing, the system was capable of detecting user movement, determining whether the user was within the operating range, recognizing predefined hand signs, and providing both visual and audio feedback.

Successful Gesture Recognition Showing Confirmed Phrase

Once a hand sign was recognized, the corresponding phrase was displayed on the OLED and spoken through the speaker.

OLED Display Showing a Recognized Phrase



The Blynk dashboard was also able to control the power state of the system, switch between male and female voice modes, and enable or disable audio playback. To give an example, users could disable audio while still retaining text output on the OLED display.

Demo Video Using SwiftSign System

Youtube: <https://youtu.be/eCV0WLtpLGA>

This video demonstrates an example conversation flow using 5 of the phrases integrated into the system. It also shows various system states and how the voice changing module works.

Summary

There were several challenges during development. The first is the communication between the ESP32-CAM, Python application, and the controller ESP32, which required extensive debugging as the devices would occasionally fail to communicate over Wi-Fi. To fix this, the communication process was repeatedly tested and debugged until stable connections could be maintained between all components.

Another challenge involved audio playback. There were several issues involving file naming and communication between the controller ESP32 and DFPlayer Mini. As a result, audio files would sometimes fail to play even though the phrase had been successfully recognized. To address this, the audio files were reorganized and the communication logic between the controller ESP32 and DFPlayer Mini was revised until audio playback became consistent.

Lastly, the gesture recognition required a lot of fine tuning to work as intended. In the first iteration, several hand signs were not distinct enough and were often mistaken for different phrases. This was especially noticeable when different finger combinations produced similar landmark patterns. To address this, multiple gestures were redesigned and a prediction buffer and phrase lock mechanism were implemented to improve reliability.

Overall, the prototype was able to achieve its intended objective of creating a simple communication tool for classroom environments using hand gesture recognition and IoT technologies. Although the current system only supports eight predefined phrases and still requires a laptop to execute the Python script, it demonstrates that relatively inexpensive hardware and computer vision technologies can be combined to create an assistive communication device.

In conclusion, there is still much room for improvement particularly in the way it recognizes language. Future versions of the project would expand the system beyond predefined

phrases to support actual recognition of sign languages, such as ASL or TSL. This could be achieved by collecting more hand sign data and training machine learning models to recognize more complex hand patterns and movements. The hardware could also be improved by implementing a Raspberry Pi to house the gesture recognition component. This would eliminate the need for a laptop, making the overall system smaller and easier to deploy.

References

- Blynk. (2025). Blynk documentation. <https://docs.blynk.io>
- Espressif Systems. (2025). ESP32 series datasheet. <https://www.espressif.com/en/support/documents/technical-documents>
- Ismail, I., Yasin, S. N. T. M., Ismail, I., & Zaini, R. E. (2023). Designing assistive devices to support learning for students with hearing impairments. *International Journal of Advanced Natural Sciences and Engineering Researches*, 7(6), 311–317. <https://doi.org/10.59287/ijanser.1168>
- Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C. Y., Yong, M., Lee, J., Chang, W. T., Hua, W., Georg, M., & Grundmann, M. (2019). MediaPipe: A framework for building perception pipelines. arXiv. <https://arxiv.org/abs/1906.08172>
- Maashi, M., Iskandar, H. G., & Rizwanullah, M. (2025). IoT-driven smart assistive communication system for the hearing impaired with hybrid deep learning models for sign language recognition. *Scientific Reports*, 15, Article 6192. <https://doi.org/10.1038/s41598-025-89975-1>